

06/22/25 - 1.5.3 - Time Complexity of while and if

For loop and while loop are basically the same thing, whatever you can do with a for you do with while and vice versa. *

While Loop

```
while (condition)
{
    statement;
}
```

Do While Loop

```
do
{
    statement;
}
while (condition)
```

repeats as long as condition is true

Repeat Until Loop

```
repeat
{
    statement;
}
until (condition)
```

repeats as long as condition is FALSE

repeat until the condition happens basically.

$i = 0 \rightarrow 1$
 $\text{while } (i < n) \rightarrow n+1$
{
 statement; $\rightarrow n$
 $i++$; $\rightarrow n$
}

the condition is true n times and one time its false. so $n+1$. if $n = 10$, runs from 0 to 9 (10 times) and then one extra so 11.

$3n+2 \rightarrow \boxed{O(n)}$

Video 1.5.3 Continued

$a = 1 \longrightarrow 1$

while ($a < b$)

```
{
  statement;
  a = a * 2;
}
```

now b/c there's no n here, we need to approach this differently.

$a < b$ we need to trace this

	<u>a</u>
	1
$1 \times 2 = 2$	$= 2$
$2 \times 2 = 4$	$= 2^2$
$4 \times 2 = 8$	$= 2^3$
	$\vdots$
	2^k

so $a = 2^k$

and the condition is false until

$a \geq b$

$2^k \geq b$

$2^k = b$

$k = \log_2 b$

$O(\log_2 n)$ or $O(\log n)$

```
i = n
while (i > 1)
{
  statement;
  i = i/2;
}
```

b/c this is $i = i/2$

we know based on the

formula this is $O(\log n)$

Video 1.5.3 Continued

```
i = 1;
k = 1;
while (k < n)
{
    statement;
    k = k + i;
    i++;
}
```

need to trace b/c
of k and i being updated
and its not straight
forward

False when $k \geq n$

$$\frac{n(n+1)}{2} \geq n$$

$$n^2 = n$$

$$n = \sqrt{n}$$

so $\boxed{O(\sqrt{n})}$

i	k
1	1
2	2 $\rightarrow 1+1$
3	4 $\rightarrow 2+2$
4	7 $\rightarrow 4+3$
$\vdots$	$\vdots$

theres no clear
pattern here, so
we just say this
goes on for m
times

$$1 + 2 + 4 + 7 + \dots + m$$

sum of natural numbers

so $\frac{n(n+1)}{2}$

NOTE: - So if we have a condition that doesn't involve n, we need to trace it. And even when its a complex condition, we should trace it.

- The condition is very important for loops of any kind!!

Types of Time Functions

$O(1)$ — constant $f(n) = 2$
 $f(n) = 5000 \rightarrow O(1)$

$O(\log n)$ — logarithmic $f(n) = i/2$
 $f(n) = i = i * 2$

$O(n)$ — linear $f(n) = 2n + 3$
 $f(n) = 3n + 9000$

$O(n^2)$ — quadratic (matrix multiply)

$O(n^3)$ — cubic (matrix add)

$O(2^n)$ — exponential

06/22/25 - Video 1.7 - Compare Class of Functions

★ This is the most important factor when dealing with time complexity. ★
we want to find the best time complexity and we can ultimately classify it from best to worst ★

$$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

absolute best

Constant

logarithmic

square root

linear

linearithmic

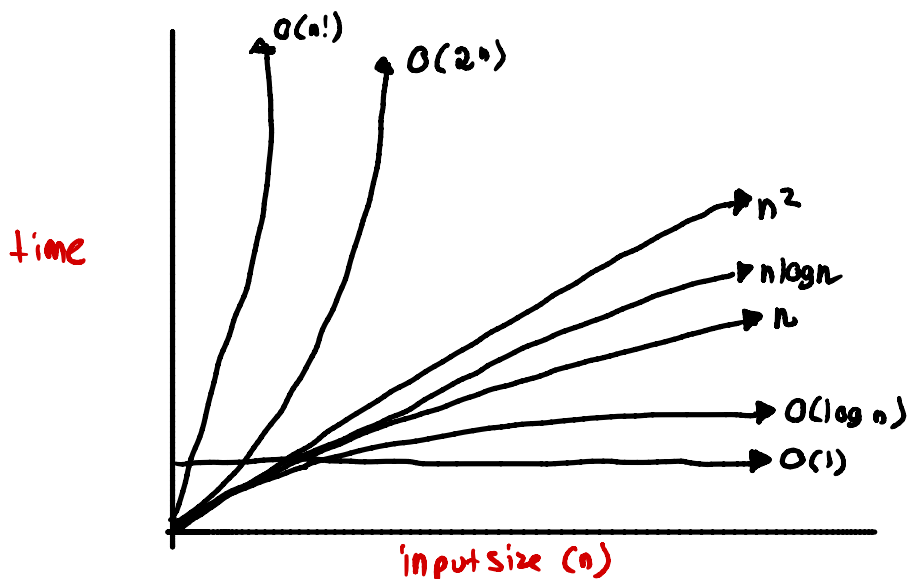
Quadratic

cubic

Exponential

factorial

absolute worst



From the diagram, we see that $O(n!)$ and $O(2^n)$ are the worst, they take too much time.